

Candidate Project Submission

Plug-and-Play Sensing & Control Slice for a Growing Rack

Joycelyn Wirata

July 21, 2026

Contents

1	Component Selection Matrix	3
1.1	Design	3
1.2	Temperature Sensor Evaluation	3
1.3	Humidity Sensor Evaluation	4
1.4	Exhaust Fan Evaluation	4
1.5	Selected Components	5
1.6	Weakest Buy vs. Build Decision	6
2	Integration and Control Design	6
2.1	System Architecture	6
2.2	Home Assistant Integration	7
2.3	Non-Native Sensor Bridge	8
2.4	Correlation Logic	9
2.5	Failure Modes	9
2.6	Bench Testing	10
2.7	Architectural Alignment	10
3	Working Configuration	10
3.1	Home Assistant Artifacts	10
3.2	Dashboard Sketch	11
4	AI Usage Appendix	11
4.1	Tools Used	11
4.2	Important Prompts	12

4.3	AI Verification	12
4.4	Work Not Delegated	12
5	Time Allocation	12
A	Component Links	13
A.1	Temperature Sensors	13
A.2	Humidity Sensors	13
A.3	Exhaust Fans	13
A.4	Control Hardware	13

1 Component Selection Matrix

1.1 Design

The proposed sensing and control slice monitors two environmental parameters—air temperature and relative humidity—and controls a single exhaust fan. Home Assistant combines both measurements to calculate Vapor Pressure Deficit (VPD), which serves as the control variable for rack ventilation.

Environmental Parameters	Air Temperature, Relative Humidity
Actuator	Exhaust Fan
Control Variable	Vapor Pressure Deficit (VPD)
Home Assistant Role	Sensor fusion, automation, actuator control

Table 1: Selected sensing and control architecture.

1.2 Temperature Sensor Evaluation

Product	Vendor	Price (USD)
Adafruit TMP117	Adafruit	\$11.50
Adafruit MCP9808	Adafruit	\$4.95
DFRobot Gravity DS18B20	DFRobot	\$6.90

Table 2: Temperature sensor candidates

Product	HA	Data	Reliability	Cost	Availability	Reasoning
Adafruit TMP117	5	5	4	3	5	$\pm 0.1^{\circ}\text{C}$ accuracy, I ² C interface, highest accuracy of the three.
Adafruit MCP9808	5	4	4	5	5	$\pm 0.25^{\circ}\text{C}$ typical accuracy, I ² C interface, lowest cost.
DFRobot Gravity DS18B20	5	3	4	4	5	$\pm 0.5^{\circ}\text{C}$ accuracy, 1-Wire interface, waterproof probe available.

Table 3: Temperature sensor evaluation scores (5 = best).

1.3 Humidity Sensor Evaluation

Product	Vendor	Price (USD)
Adafruit HTU21D-F	Adafruit	\$10.95
DFRobot SHT20	DFRobot	\$22.50
Seeed Grove SHT40	Seeed Studio	\$5.50

Table 4: Humidity sensor candidates

Product	HA	Data	Reliability	Cost	Availability	Reasoning
Adafruit HTU21D-F	5	4	4	4	5	$\pm 2\%$ RH accuracy, I ² C interface, well-supported by ESPHome.
DFRobot SHT20	5	3	4	2	5	$\pm 3\%$ RH accuracy, I ² C interface, highest cost of the three.
Seeed Grove SHT40	5	5	4	5	5	$\pm 1.8\%$ RH accuracy, I ² C interface, highest accuracy and lowest cost.

Table 5: Humidity sensor evaluation scores (5 = best). Scores are based on Home Assistant integration (via ESPHome), data quality, suitability for unattended operation, cost, and availability.

1.4 Exhaust Fan Evaluation

Product	Vendor	Price	Link
AC Infinity AXIAL 1238	AC Infinity	\$39	AC Infinity
AC Infinity CLOUDLINE S6	AC Infinity	\$100	AC Infinity
VIVOSUN AeroZesh G6	VIVOSUN	\$104	VIVOSUN

Table 6: Exhaust fan candidates.

Product	Control	Reliability	Noise	Cost	Availability
AC Infinity AXIAL 1238	5	5	4	5	5
AC Infinity CLOUDLINE S6	5	5	5	2	5
VIVOSUN AeroZesh G6	5	4	4	2	5

Table 7: Exhaust fan evaluation scores (5 = best).

1.5 Selected Components

Component	Selected Device	Reason
Temperature Sensor	Adafruit TMP117	Highest temperature accuracy ($\pm 0.1^\circ\text{C}$), I ² C interface, ESPHome compatible.
Humidity Sensor	Seeed Grove SHT40	Highest humidity accuracy ($\pm 1.8\%$ RH), I ² C interface, ESPHome compatible.
Exhaust Fan	AC Infinity AXIAL 1238	Low cost, continuous-duty operation, suitable for simple Home Assistant control.

Table 8: Selected components.

Selected Temperature Sensor: Adafruit TMP117 The Adafruit TMP117 was selected because it provides the highest temperature accuracy ($\pm 0.1^\circ\text{C}$) among the evaluated sensors while maintaining compatibility with ESPHome through its I²C interface. Although it is the most expensive option, its measurement performance makes it the preferred choice for accurate VPD calculations.

Selected Humidity Sensor: Seeed Grove SHT40 The Seeed Grove SHT40 was selected because it offers the highest humidity accuracy ($\pm 1.8\%$ RH) of the evaluated sensors while also being the lowest-cost option. Its I²C interface is natively supported by ESPHome, making integration with Home Assistant straightforward.

Selected Exhaust Fan: AC Infinity AXIAL 1238 The AC Infinity AXIAL 1238 was selected because it provides reliable continuous ventilation at the lowest cost among the evaluated fans. It can be easily controlled through a Matter-compatible smart plug, allowing all automation logic to remain centralized within Home Assistant.

1.6 Weakest Buy vs. Build Decision

The weakest buy in the proposed design is the humidity sensor. While the Seeed Grove SHT40 provides excellent measurement accuracy and integrates cleanly with ESPHome, it is still a standalone module that requires an external microcontroller for Home Assistant integration.

For a single rack or small deployment, purchasing the sensor is the most practical approach and aligns with Luce Labs’ “buy before build” philosophy. However, for large-scale deployments, it would be more cost-effective to develop a custom sensing board that integrates the humidity sensor with the required electronics and communication interface. This would reduce wiring complexity, simplify installation, and allow the sensing node to expose the same Home Assistant entities while remaining compatible with the existing automation logic.

2 Integration and Control Design

2.1 System Architecture

Figure 1 shows the sensing and control slice for one growing rack. The Adafruit TMP117 measures air temperature, while the Seeed Grove SHT40 measures relative humidity. Both sensors communicate with an ESP32-DEVKITM-1 over the I²C bus using ESPHome. The ESP32 publishes sensor data to Home Assistant over WiFi. Home Assistant calculates Vapor Pressure Deficit (VPD), executes the automation logic, and controls the AC Infinity AXIAL 1238 exhaust fan through a Matter command sent to the Shelly Plug US Gen4.

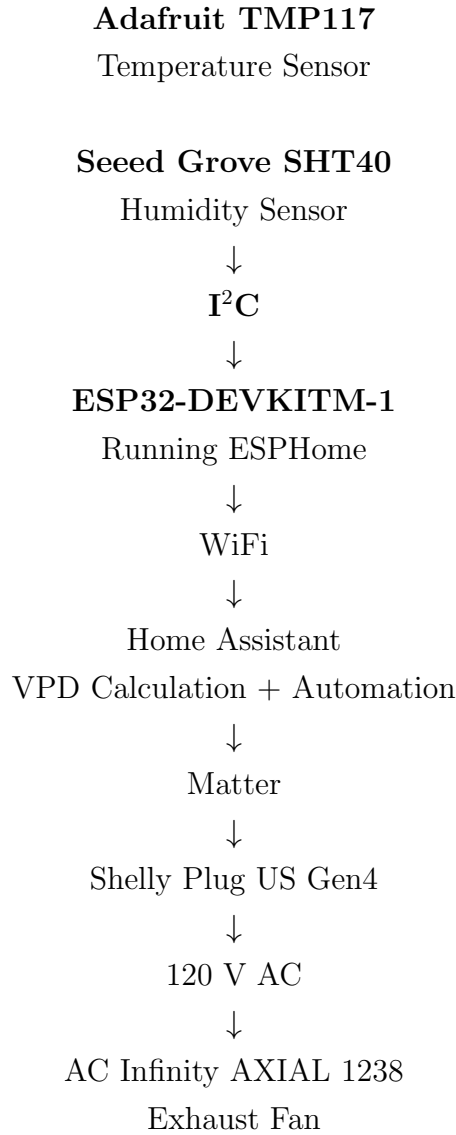


Figure 1: Proposed sensing and control architecture.

2.2 Home Assistant Integration

Temperature Sensor

The Adafruit TMP117 communicates with the ESP32 using the I²C interface. ESPHome publishes the following entity to Home Assistant:

- `sensor.rack_temperature`

Humidity Sensor

The Seeed Grove SHT40 also communicates with the ESP32 using I²C. ESPHome publishes:

- `sensor.rack_humidity`

Both entities are automatically discovered through the native ESPHome integration.

ESP32 Bridge

The ESP32-DEVKITM-1 runs ESPHome firmware and connects to Home Assistant through native integration. ESPHome provides automatic entity discovery and OTA firmware updates without requiring MQTT middleware.

Smart Plug

The Shelly Plug US Gen4 joins Home Assistant using the Matter integration and exposes the following entities:

- `switch.rack_fan`
- `sensor.rack_fan_power`
- `sensor.rack_fan_energy`

The smart plug acts solely as the actuator interface. All environmental decision making remains centralized within Home Assistant.

2.3 Non-Native Sensor Bridge

The Adafruit TMP117 and Seeed Grove SHT40 are not native Matter or Home Assistant devices, so the ESP32-DEVKITM-1 acts as the hardware and firmware bridge. ESPHome reads both sensors over I²C and publishes them as standard Home Assistant entities.

Home Assistant interacts only with the exposed entities, making the sensing node functionally identical to a native HA device. The sensor hardware can therefore be replaced without modifying dashboards or automation logic.

Bridge Selection

Option	Reason Not Selected
MQTT Discovery	Requires an MQTT broker and additional configuration overhead.
Matter Bridge	Adds unnecessary complexity for a simple I ² C sensor node.
ESPHome	Native Home Assistant support, automatic entity discovery, OTA firmware updates, and minimal configuration.

Table 9: Comparison of sensor bridging approaches.

2.4 Correlation Logic

Home Assistant combines temperature and relative humidity measurements to calculate Vapor Pressure Deficit (VPD). VPD provides a more meaningful indicator of plant transpiration conditions than either parameter alone.

The exhaust fan is controlled according to the following thresholds.

Condition	Action
VPD > 1.2 kPa for 5 minutes	Turn exhaust fan ON
VPD < 1.0 kPa for 10 minutes	Turn exhaust fan OFF

Table 10: VPD control thresholds.

The separation between ON and OFF thresholds introduces hysteresis, preventing rapid switching. Minimum activation times reduce short cycling and extend actuator lifetime.

2.5 Failure Modes

Failure Mode	Detection	Response
Temperature sensor offline	Entity unavailable	Turn fan ON and generate Home Assistant alert.
Humidity sensor offline	Entity unavailable	Turn fan ON and generate Home Assistant alert.
Implausible sensor reading	Temperature/RH outside expected operating range or unchanged for extended period	Ignore invalid data, generate alert, and maintain fan ON as a safe state.
ESP32 offline	Device unavailable	Turn fan ON and notify operator.
Fan commanded ON but drawing 0 W	Shelly power monitoring	Generate maintenance alert indicating possible actuator failure.
Home Assistant restart	Automation reload	Re-evaluate sensor states and VPD, then restore appropriate fan state.
WiFi loss between the ESP32 and Home Assistant	Sensor entities become unavailable	Home Assistant commands the fan ON if the Shelly Plug remains reachable and generates an alert.

Table 11: Failure modes and safe-state responses.

The proposed design adopts a ventilation-first safe state because excessive humidity presents a greater long-term risk to unattended plant growth than continuous airflow.

2.6 Bench Testing

Before deployment, the following tests would be performed:

1. Verify TMP117 temperature measurements against a reference thermometer.
2. Verify SHT40 humidity measurements against a reference hygrometer.
3. Confirm ESPHome discovery and HA entity creation.
4. Validate VPD calculations using known temperature/RH values.
5. Test fan activation, hysteresis, and anti-short-cycling behavior.
6. Disconnect sensors and verify safe-state behavior.
7. Test WiFi loss and Home Assistant restart recovery.
8. Perform a 24-hour continuous operation test.

2.7 Architectural Alignment

This design aligns with Luce Labs’ plug-and-play architecture by keeping Home Assistant as the central control layer and using open protocols including I²C, WiFi, ESPHome, and Matter. The ESP32 bridge converts a non-native sensor into a first-class Home Assistant device while preserving hardware flexibility and scalability.

3 Working Configuration

3.1 Home Assistant Artifacts

The Home Assistant configuration is organized into separate files for sensor integration, data processing, automation, and visualization.

- `esphome/rack-sensor.yaml`: ESPHome firmware configuration for the ESP32 bridge, TMP117 temperature sensor, and SHT40 humidity sensor.
- `template_sensors.yaml`: Creates the VPD calculation entity from temperature and humidity measurements.
- `automations.yaml`: Implements the VPD-based exhaust fan control logic with hysteresis and anti-short-cycling.

- Dashboard sketch: Visual representation of the Home Assistant Lovelace dashboard layout.

3.2 Dashboard Sketch

The dashboard provides operators with a real-time overview of environmental conditions, actuator status, and system health.

+-----+		
	Growing Rack Dashboard	
+-----+		
	Temperature: 24.3 °C	
	Humidity: 62 %	
	VPD: 1.08 kPa	
+-----+		
	Exhaust Fan: OFF	
	Fan Power: 0 W	
	Energy Usage: 0.08 kWh	
+-----+		
	System Status	
	ESP32 Bridge: Online	
	TMP117 Sensor: Connected	
	SHT40 Sensor: Connected	
	Shelly Plug: Connected	
	Home Assistant: Running	
+-----+		
	Historical Data	
	Temperature / Humidity / VPD Trends	
+-----+		

4 AI Usage Appendix

AI tools were used to accelerate component research, Home Assistant configuration development, and document refinement.

4.1 Tools Used

- ChatGPT: Used for architecture brainstorming, ESPHome YAML drafting, and checking Home Assistant configuration consistency.
- Web search: Used to verify product availability, pricing, and protocol support.

4.2 Important Prompts

Example prompts used:

1. "Design a Home Assistant architecture using a non-native I2C sensor bridged through ESPHome."
2. "Generate a Home Assistant automation using VPD thresholds with hysteresis."
3. "Compare Matter smart plugs suitable for Home Assistant."

4.3 AI Verification

One issue encountered was that AI-generated suggestions occasionally referenced unsupported Home Assistant integrations or inaccurate product capabilities. These outputs were verified against vendor documentation before inclusion.

For example, product protocol support and Home Assistant compatibility were manually checked before selecting the Shelly Plug US Gen4.

4.4 Work Not Delegated

The final architecture decisions, product selection, tradeoffs, and failure-mode analysis were completed manually. AI was used as a productivity tool rather than replacing engineering judgment.

5 Time Allocation

The project was completed within the 6–8 hour time constraint.

Task	Time Spent
Component research and evaluation	2 hours
Architecture design and block diagram	1 hour
Home Assistant and ESPHome configuration	2 hours
Failure analysis and testing plan	1 hour
Documentation and review	1 hour
Total	7 hours

Table 12: Project time allocation.

A Component Links

The following links correspond to the products evaluated in the component selection matrix.

A.1 Temperature Sensors

- Adafruit TMP117: [Adafruit TMP117](#)
- Adafruit MCP9808: [Adafruit MCP9808](#)
- DFRobot Gravity DS18B20: [DFRobot Gravity DS18B20](#)

A.2 Humidity Sensors

- Adafruit HTU21D-F: [Adafruit HTU21D-F](#)
- DFRobot SHT20: [DFRobot SHT20](#)
- Seeed Grove SHT40: [Seeed Grove SHT40](#)

A.3 Exhaust Fans

- AC Infinity AXIAL 1238: [AC Infinity AXIAL 1238](#)
- AC Infinity CLOUDLINE S6: [AC Infinity CLOUDLINE S6](#)
- VIVOSUN AeroZesh: [VIVOSUN AeroZesh G6](#)

A.4 Control Hardware

- ESP32-DEVKITM-1: [DigiKey ESP32-DEVKITM-1](#)
- Shelly Plug US Gen4: [Shelly Plug US Gen4](#)